

Introduction to Linux for HPC

assoc. prof. dr. Biljana Mileva Boshkoska

Faculty of information studies in Novo mesto

October, 2022

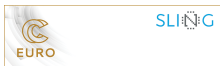


Table of contents I

Introduction to operating systems

Linux operating system

- Linux operating system overview
- Linux kernel
- Linux distribution
- Ubuntu Linux
- Bash

Part 1: Basic Commands

- Linux commands

Part 2: Commands for working with directories

- Working with directories

Permissions

- Owners, group members, everyone else
- Command to change file permissions chmod

Manage files and directories

- Creating text files
- Creating directories
- Copy files and directories
- Moving and renaming files and directories
- Delete files and directories
- Wildcard

Part 3: Diversion and pipeline

- Standard output
- Standard error
- Standard input
- Pipelines
- Filters

Exercises

Table of contents II

Basics of shell programming

- Shell scripts

- Message display – echo command

- Variables

Using structured commands

- The structure if–then–else

SSH in X2go

- Reference



What is an operating system?

- ▶ *Operating system* (OS) is software that manages hardware.
- ▶ Main tasks:
 - ▶ Interface between user level and *computer resources*
 - ▶ Interaction and communication with users
 - ▶ Resource management
 - ▶ A set of useful service routines
 - ▶ A set of tools to develop and manage projects

Basic functionalities of operating systems

- ▶ Process management
- ▶ Device management
- ▶ Memory management
- ▶ File management
- ▶ Network management

Introduction to Linux

- ▶ Linux is a variant of the UNIX operating system
- ▶ UNIX is one of the oldest types of OS that provides reliability and security in professional programs for nearly half a century.
- ▶ Many servers around the world that store data for popular sites (such as YouTube and Google) are powered by versions of Unix.
- ▶ Unix was completely based on the command line interface until the early 1990s, when graphical user interfaces began to appear.
- ▶ Open source and free system software

Historical development of Linux

- ▶ Linus Torvalds
- ▶ The first version of Linux 0.01 from 1991
 - ▶ uses many ideas from the MINIX system, which is a UNIX system and is based on a microkernel design
 - ▶ monolithic system, 9300 lines of code in C programming language and 950 lines of code in assembler
 - ▶ virtual memory, more sophisticated collection system ...
- ▶ Linux 1.0 from 1994
 - ▶ 165 000 lines of code in C, new collection system, memory map files, TCP / IP protocols, drivers ...
- ▶ Linux 2.0 from 1996
 - ▶ 470 000 lines of code in C, 8000 lines of code in assembler, support for 64-bit architectures, symmetric multiprocessing, new network protocols, a large number of device drivers

Linux kernel (textit linux kernel)

- ▶ The Linux kernel is the operating system controller.
- ▶ The Kernel is responsible for memory and processor time allocation.
- ▶ You can think about it as a program that manages all the programs on your computer.
- ▶ Format of the kernel version A.B.C.D (example 2.6.9.11)
 - ▶ A - main kernel version number
 - ▶ B - major revision number
 - ▶ C - minor revision number
 - ▶ D - very small change number related to bug fix or security patch

Linux distributions

- ▶ Linux distribution - an operating system that is built on the basis of the Linux kernel and uses a package management system.
- ▶ The distribution includes a large collection of diverse open source and free software organized in the form of packages.
- ▶ There are currently more than 600 different Linux distributions (of which about 300 are in active development)
- ▶ Popular distributions
 - ▶ Debian - a non-commercial distribution maintained by a voluntary group of developers
 - ▶ **Ubuntu - a popular desktop and server distribution derived from Debian**
 - ▶ Fedora - a distribution sponsored by the American company Red Hat
 - ▶ Gentoo - distribution intended for more demanding users
 - ▶ Redhat - distribution intended for more demanding users
 - ▶ ...

Ubuntu Linux

- ▶ In the workshop we will use Ubuntu distribution of Linux 18.04 LTS (\$ `lsb_release -a`) kernel version 4.15.0-122-generic (\$ `uname -r`)
- ▶ Ubuntu was founded by Mark Shuttleworth and his company Canonical in 2004
- ▶ Ubuntu linux uses the GNOME desktop graphical environment
- ▶ The operating system is available online www.ubuntu.com in www.ubuntu.si (Slovenian version)
- ▶ Ubuntu Linux Help for Beginners in Slovenian <https://www.ubuntu.si>
- ▶ More information <https://help.ubuntu.com>

Advantages of using the Shell

1. Under Linux, there are some powerful tools that for all practical purposes are unavailable under Windows
2. One of these tools is something called "shell programming".
3. The most common Linux shell is named "Bash".
4. Bash shell increases the chance that your scripts will be portable between machines, distributions, even operating systems.
5. Shell programming is an art
6. Because shell programming is an art, please avoid saying, "Wow, that was a really inefficient way to do such-and-such."

Online Linux Terminal

Execute Bash Shell Online:

- ▶ GNU/Linux: launch your favorite terminal application
- ▶ macOS: launch Terminal
- ▶ Windows:
 - ▶ Windows 10: Install the Windows Subsystem for Linux (WSL)
<https://docs.microsoft.com/en-us/windows/wsl/install-win10>
 - ▶ Alternatively install Git BASH (<https://gitforwindows.org/>)
- ▶ <https://cocalc.com/>
- ▶ https://www.tutorialspoint.com/execute_bash_online.php
- ▶ https://www.onlinegdb.com/online_bash_shell

Q & A

1. Q. What is the best way to figure out which shell do I use?
2. A1. `echo $SHELL`
3. A2. `ps -p $$`

Q & A

1. Q. What is the best way to figure out which shell do I use?
2. A1. `echo $SHELL` – Prints the shell for the current user but not necessarily the shell that is currently running
3. A2. `ps -p $$` – Displays your current shell name reliably.

Q & A

1. Q. What is the best way to figure out the Unix distribution that I use?
2. A1. `$ lsb_release -a`
3. A2. `$ uname -r`

Introduction to Shell

1. There are two ways to use Shell: interactive and scripting.
 - ▶ In interactive mode, the user enters the command itself (or a short set of commands) and the result is printed.
 - ▶ In the shell script, the user enters anything - from a few lines to the entire program into a text editor, then executes the resulting text file as a script.
2. It often happens that an interactive session becomes a shell script when things get too complicated for simple interactive line entries, or because it seems generally not useful and worth keeping a special sequence.

Introduction to Shell (2)

3. In a modern Linux environment the user can have more than one shell open at a time, either by moving between a sequence of independent "virtual terminals" in a text-only environment, or by opening any number of shell windows in the X Windows environment.
4. The advantage of having more than one shell available is that one shell can be used for testing one command at a time, while another might provide a text editor for assembling single commands into a shell program.
5. I don't want to get too distribution-specific, but if you are not hosting X Windows and want more than one simultaneous shell session, with many current distributions you can switch between "virtual terminals" by pressing `Ctrl+Alt+F(n)`, `n` typically between 1 and 6.
6. In an environment that supports X Windows, simply open any desired number of command shell windows and move between them.

Basic Commands in Linux Shell

- ▶ Basic Linux shell commands
 - ▶ Shell caller
 - ▶ Change the current directory (cd)
 - ▶ Absolute path
 - ▶ Relative path
 - ▶ Browsing all files in the directory (ls)
 - ▶ Permissions to access the file
 - ▶ File type
 - ▶ Scrolling through the contents of a text file (less)

Commands

- ▶ Writing convention:

- ▶ `$ date`

- ▶ Computer response:

- ▶ `Tue Dec 23 10:52:51 PST 2003`

- ▶ Shell prompt `$`

- ▶ Command syntax

`\$command_name -option`

- ▶ Each command consists of:

- ▶ command names

case names in uppercase and lowercase (case sensitive)

example. *Echo* is not the same as *echo*

- ▶ options or arguments in the format `-x`, `x` is a character

there must be no space between `-` and `x`

- ▶ arguments can be file names, directories, or expressions, depending on the command

- ▶ We end with: ENTER

Example

- ▶ Try:
- ▶ `$ whoami`
- ▶ `root`

Example

- ▶ Try:

```
$ whoami
```

```
root
```

- ▶ Example:

```
$ ls -a OS
```

ls is a command to scroll through the contents of the directory, -a is an option to display a list of all files; The OS is an argument that specifies the directory whose contents will be displayed.

- ▶ A single command can have several options that define how the command will be executed.

```
$ ls -a -l OS
```

```
$ ls -al OS
```

```
$ ls -alr OS
```

Example

- ▶ Help with a command

```
$ man command_name
```

Example:

```
$ man ls
```

- ▶ Command to display the current date and time

```
$ date
```

- ▶ Calendar display command

```
$ cal
```

- ▶ Command to end the current session (ang. session)

```
$ exit
```

Working with directories

Working with directories

- ▶ In Linux, there is a hierarchical directory structure in which files are organized.
- ▶ At the top of the structure is a special directory called root.
`$ pwd`
`/path/path/path`
- ▶ A directory is a file that contains a list of other files and does not contain other data.
- ▶ Each system user has its own home directory. How to get to your directory?
`$ cd ~`
`$ pwd` (Command to display the current directory)
`/home/username`
- ▶ Različica: `$ cd /home/username`
- ▶ When logging in to the system, the current directory becomes the home directory.

Changing the current directory - the absolute path

- ▶ Command to change the current directory

```
$ cd pot_do_imenika
```

- ▶ Absolute pathname
- ▶ The absolute path starts with the root directory, followed by a list of directories by structure to the desired directory.
- ▶ Example: on a Linux system, there is a directory that contains system programs. The absolute path to this directory is:

```
/usr/bin
```

This is the primary directory of executable commands on the system.
This directory is accessed as follows:

```
$ cd /usr/bin
```

- ▶ `/sbin`
 - Binaries needed for booting, low-level system repair, or maintenance (run level 1 or S)
- ▶ `/bin`
 - Binaries needed for normal/standard system functioning at any run level.
- ▶ `/usr/bin`
 - Application/distribution binaries meant to be accessed by locally logged in users
- ▶ `/usr/sbin`
 - Application/distribution binaries that support or configure stuff in `/sbin`
- ▶ `/usr/share/bin`
 - Application/distribution binaries or scripts meant to be accessed via the web, i.e. Apache web applications

► `*local*`

- Binaries not part of a distribution; locally compiled or manually installed. There's usually never a

`/local/bin`

but always a

`/usr/local/bin`

`/usr/local/bi`

Changing the current directory – relative path

- ▶ Relative path (relative pathname)
 - ▶ starts from the current directory
 - ▶ we use special symbols. (dot) and .. (double dot)
 - ▶ . refers to the current directory
 - ▶ .. refers to the parent directory (nickname)
- ▶ Example: we are in the directory `/usr/bin` and we want to change the current directory to `/usr` using a relative path

```
$ cd ..
```

- ▶ Example: we are in the directory `/usr` and we want to change the current directory to `/usr/bin` using a relative path

```
$ cd ./bin
```

Directories

- ▶ The name is allowed to use: lowercase and uppercase letters (case sensitive), numbers and some special characters such as `_ # @`
- ▶ The name is forbidden to use: space, metacharacters `* ? < > | / ; & ! [ ] $ ' "`, must not start with `+ and -`
- ▶ Directory whose names begin with a dot `.` are hidden files.
- ▶ `$ ls`
shows all files from the current directory (no hidden files)
- ▶ Hidden files are visible with the command:

```
$ ls -a
```

Command to list all files in the directory *ls*

- ▶ `$ ls absolute_path`
shows all files from the directory specified by the absolute path
- ▶ `$ ls -s`
shows the contents of the current directory and the file size in kilobytes
- ▶ `$ ls -l`
displays the current directory files in a long format that includes permissions for each file, size, and creation date.
- ▶ Options can be grouped:
`$ ls -la` (all files, one file per line)

Task

Try and consider what does the following command:

\$ ls -ltr

Hint: You can check the command options with: **\$man ls**

Example ls -l

1. File type and permission to access the file
2. Number of links to the file
3. Owner of the file
4. File size in bytes
5. Last file modification time
6. File name

Permissions

- ▶ Linux operating system is a multi-user system
- ▶ More than one user can use the computer at the same time
- ▶ There are methods to protect users

Owners, group members, everyone else

- ▶ In Linux, users can own files and directories
- ▶ The owner of a file or directory has access control
- ▶ Users can also be members of groups that have been given the right to access files and directories by their owners
- ▶ The owner of the file and directory can grant access rights to all users
- ▶ The *id* command displays user identity information

```
$ id
```

- ▶ Identifiers
 - ▶ uid - user identifier, usually mapped to username
 - ▶ gid - user's real primary group ID
 - ▶ groups - real IDs of the supplemental groups the user belongs to.

File type and access permission

- ▶ File type
 - ▶ - - ordinary directory
 - ▶ d directory
 - ▶ l link
- ▶ File users
 - ▶ user - file owner access rights
 - ▶ group - access rights of users who are members of the same group as the file owner
 - ▶ other - file access rights for all other users
- ▶ Different access permits
 - ▶ r - read permission
 - ▶ w - write permission
 - ▶ x - permission to execute

Permissions for files and directories

- ▶ r – read permission
 - ▶ permission to read the file
 - ▶ permission to print the contents of the directory
- ▶ w – write permission
 - ▶ permission to change the contents of the file
 - ▶ permission to change directory contents (add and delete subdirectories and files)
- ▶ x – execute permission
 - ▶ permission to execute the file
 - ▶ directory permission to become the current directory

Command to change file permissions chmod

- ▶ The chmod command is used to change the permissions of a file or directory.
- ▶ The change can only be made by the file owner and the so-called superuser
- ▶ The chmod command supports two different ways to change the permission: by using octal numbers and by using symbols

Symbolic notation

- ▶ For now, we will focus on using symbols to change the permission. The notation is divided into 3 parts:
 - ▶ to whom the change applies
 - ▶ u (user, owner)
 - ▶ g (group)
 - ▶ o (all others)
 - ▶ a (all combination u g o)
 - ▶ which operation will be performed
 - ▶ + (add permission)
 - ▶ - (withdraw permission)
 - ▶ = (only special permissions will be executed, all others will be revoked)
 - ▶ which permission will be set
 - ▶ r (read)
 - ▶ w (record)
 - ▶ x (execution)

Examples symbolic notations for the command chmod

Notation	Meaning
u + x	Add Execution Permission to User
u-x	Undo Execution Permission
+ x	Add execution permission to user, group, and everyone else. Equivalent to the notation a + x
o-rw	Revoke read and write permissions for anyone except the owner and group owner

Example: `$chmod +x ime_datoteke.sh`

Command for file type

► `$ file file_name`

► Example:

```
$ file picture.jpg
```

```
picture.jpg: JPEG image data
```

```
$ file tux_small.png
```

```
tux_small.png: PNG image data, 128 x 151, 8-bit/color RGB,  
non-interlaced
```

Command to print the contents of a text file

▶ `$ less ime_datoteke`

▶ Example:

`$ less /etc/passwd`

File and directory management

- ▶ File and directory management commands
 - ▶ Copying files and directories (*cp*)
 - ▶ Move and rename files and directories (*mv*)
 - ▶ Creating directories (*mkdir*)
 - ▶ Delete files and directories (*rm*)
- ▶ Why use command line commands when we already have a graphical interface in Linux for these same operations?
 - ▶ Power and flexibility
 - ▶ Complex tasks are executed faster than in a graphical interface
 - ▶ Example: How would you copy all HTML files from one directory to another, but only those that do not already exist in the target directory or are newer than the versions of the files in the target directory?
 - ▶ A very difficult task to perform in a graphical interface
 - ▶ In the command line interface:

```
$ cp -u * .html destination
```

Command for creating text files (touch)

The touch command is a standard command used in UNIX/Linux operating system which is used to create, change and modify timestamps of a file. Basically, there are two different commands to create a file in the Linux system which is as follows:

- ▶ `$ cat >newfile`

Will create a new file

`$ ls newfile`

- ▶ cat command is used to create the file with content.

`$ touch file_name`

- ▶ touch command is used to create a file without any content. The file created using touch command is empty. This command can be used when the user does not have data to store at the time of file creation.

`$touch file_name_1 file_name_2 file_name_3`

Command for creating directories (mkdir)

► `$mkdir directory...`

► Example – command to create one directory named directory1

```
$ mkdir directory1
```

► Example – command to create directories named directory1, directory2, directory3

```
$mkdir directory1 directory2 directory3
```

► The created directories are subdirectories of the current directory

Command to copy files and directories (cp)

- ▶ Copy one file or directory named file1 to a file2 or directory named file2

```
$ cp file1 file2
```

- ▶ Copy multiple files to another directory

```
$ cp file ... directory
```

Command options: cp

Option	Meaning
−a	Copy files and directories including all attributes and permissions. Copies are given the attributes and permissions of the copying user.
−i	Before overwriting existing files, prompt the user to confirm.
−r	Recursively copy directories and their contents. This option is used when copying directories if we want all files and subdirectories of the directory we are copying to be copied as well.
−u	When copying files from one directory to another, copy only files that do not exist or are newer than existing files in the target directory.
−v	Show informational messages while copying.

Examples

- ▶ `$cp file1 file2`
- ▶ `$cp -i file1 file2`
- ▶ `$cp file1 file2 directory1`
- ▶ `$cp directory1/* directory2`
- ▶ `$cp -r directory1 directory2`

Command to move and rename files and directories (mv)

- ▶ Move or rename one file or directory named file1 to a file2 or directory named directory2

```
$ mv file1 file2
```

- ▶ Moving multiple things from one directory to another

```
$ mv file... directory
```

Command options: mv

Option	Meaning
-i	Before overwriting existing files, prompt the user to confirm.
-u	When moving files from one directory to another, move only files that do not exist or are newer than existing files in the target directory.
-v	Show informational messages while moving.

Examples

- ▶ `$mv datoteka1 datoteka2`
- ▶ `$mv -i datoteka1 datoteka2`
- ▶ `$mv datoteka1 datoteka2 imenik1`
- ▶ `$mv imenik1 imenik2`

Command to delete files and directories (rm)

- ▶ Delete one or more files and directories

```
$ rm stvar...
```

Command options: rm

Option	Meaning
-i	Before deleting an existing file, prompt the user to confirm. If this option is not specified, the command will silently delete the file.
-r	Recursively delete directories and their contents. This option is used when deleting directories if we also want to delete all files and subdirectories of the directory we are deleting.
-f	Ignore a non-existent file and do not prompt the user to confirm. This overrides the -i option.
-v	Show informational messages while deleting.

- └─ Manage files and directories
- └─ Delete files and directories

Examples

- ▶ `$rm datoteka1`
- ▶ `$rm -i datoteka1`
- ▶ `$rm -r datoteka1 imenik1`
- ▶ `$rm -rf datoteka1 imenik1`

Wildcard

- ▶ Wildcards are special characters that help you quickly identify groups of names
- ▶ Allows you to select file names based on patterns of characters
- ▶ Wildcards can be used with any command that accepts file names as an argument (example ls)

List of wildcards

Wildcard	meaning
*	Matches any character
?	Matches a single character
[<i>characters</i>]	Matches any character that is a member of the character set
[! <i>characters</i>]	Matches any character that is not a member of the character set
[[<i>: class :</i>]]	Matches any character that is a member of a particular class ([[<i>: alnum :</i>]], [[<i>: alpha :</i>]], [[<i>: digit :</i>]], [[<i>: lower :</i>]], [[<i>: upper :</i>]])

Examples with wildcard

Sample	Meaning
<code>*</code>	all files
<code>g *</code>	any file starting with the letter g
<code>b * .txt</code>	any file beginning with the letter b followed by any character ending in .txt
<code>Date ???</code>	any file that starts with Data and is followed by exactly three characters
<code>[abc]*</code>	any file starting with a or b or c
<code>* b. [0 - 9] [0 - 9] [0 - 9]</code>	any file starting with b. followed by 3 digits
<code>[[: upper :]]*</code>	any file starting with a capital letter
<code>[[! [: digit :]]*</code>	any file that does not start with a digit
<code>* [[: lower :] 123]*</code>	any file starting with a lowercase letter or digits 1 or 2 or 3

Diversion and pipeline

- ▶ I / O redirection - redirects input and output commands to and from files
- ▶ Pipelines - connecting several commands together into a stronger command
- ▶ We will demonstrate the diversion and pipe functionalities via the following commands:
 - ▶ Linking files (*cat*)
 - ▶ Sorting text swirls
 - ▶ Notify or omit duplicate lines (*uniq*)
 - ▶ Print lines that match the pattern (*grep*)
 - ▶ Print the number of lines, words and bytes for each file (*wc*)
 - ▶ Print the first part of the file (*head*)
 - ▶ Print the rest of the file (*tail*)

Standard input, standard output and standard error

- ▶ Most of the programs we have used so far have been used with the intention of producing some kind of output. There are two types of exits:
 - ▶ Program results
 - ▶ Status messages and error messages
- ▶ Example: The `ls` command displays program results and error messages on the screen

Everything is a file

- ▶ Unix / Linux philosophy - everything is a file
 - ▶ Standard output - *stdout*
 - ▶ Standard error - *stderr*
 - ▶ Standard input - *stdin*
- ▶ The default standard output and standard error is the display
- ▶ The default standard input is the keyboard
- ▶ Input and output redirection allows us to change where the output goes and where the input comes from.

Standard output redirection

- ▶ Use the ">" operator followed by the name of the file to which we are redirecting the output. For example:

```
$ ls -l /usr/bin > ls-output.txt
```

To view the contents of the file use:

```
$ less ls-output.txt
```

- ▶ In case we want to execute the command from the example and the /usr/bin directory does not exist, we will get a display of the error. The error will not be written to the output file and its size is 0 bytes.

Redirect to an existing file

- ▶ When redirecting with the operator ">" the output file is always overwritten from the beginning, if it already exists; if it does not exist, a new file opens.
- ▶ If we want to append the output to an existing file, we use the operator ">>"
- ▶ Example:

```
$ ls -l /usr/bin >> ls-output.txt
```

Redirect standard error

There are two main output streams in Linux (and other OSs), standard output (stdout) and standard error (stderr). Error messages are printed to standard error. The classic redirection operator (*command* > *file*) only redirects standard output, and there is no special operator to redirect the standard error, so standard error is still shown on the terminal. To redirect stderr as well, you have a few choices:

- ▶ Redirect stdout to one file and stderr to another file. A file descriptor is used for this purpose: 0 - input, 1 - output, and 2 - error. Example:

```
$ ls -l /bin/usr 2> ls-error.txt
```

- ▶ In cases where we want to redirect the entire output of the command to the same file, we need to redirect the standard output and the standard error at the same time. Redirect stdout to a file (>out), and then redirect stderr to stdout (2>&1). Example:

```
$ ls -l /bin/usr > ls-output.txt 2>&1
```

- ▶ The redirection order is very important. Standard error redirection is always after standard output redirection.

Removing unnecessary result ("silence is golden")

- ▶ In case we don't need the result of the command and we just want to catch errors and status messages, we redirect the standard output to a special file named `/dev/null`.
- ▶ Example:

```
$ ls -l /bin/usr 2> /dev/null
```


What is dev Null and How to Use It

`/dev/null`

is a special file that's present in every single Linux system. However, unlike most other virtual files, instead of reading, it's used to write.

`/dev/null`

in Linux is a null device file. This will discard anything written to it, and will return EOF on reading. This is a command-line hack that acts as a vacuum, that sucks anything thrown to it.

The command cat

- ▶ a command that reads one or more files and copies them to standard output
- ▶ `$ cat [file...]`
- ▶ Example: The cat command can be used to display small text files on the screen

```
$ cat ls-output.txt
```

- ▶ Another use of the cat command is to combine (concatenate) multiple files. For example, let us assume that we have uploaded one large file from the web, which was split into many small files, and now we would like to merge them back into one file (movie.mpeg.001 movie.mpeg.002 ... movie.mpeg.099)

```
$cat movie.mpeg.0* > movie.mpeg
```

Standard input redirection

- ▶ What happens to the cat command if we enter it without arguments?

```
$ cat
```

- ▶ In the absence of arguments, the cat command reads from standard input (keyboard), so it waits for the user to write something.

```
$ cat
```

```
This is a Linux tutorial.
```

After entering the text, press enter and CTRL - d, which tells the command that we have reached the end of the file (EOF) on the standard input.

- ▶ The command will display the same content on a standard screen.

Creating short text files

- ▶ We can use the standard input redirection option to create short text files.

```
$ cat >perica.txt
```

```
This is a Linux tutorial.
```

- ▶ Use the operator ">" for this purpose and then the name of the file without space between them.

Pipelines

- ▶ Pipelines use the ability of commands to read data from standard input and send results to standard output.
- ▶ Operator | (pipe operator)
- ▶ Using the operator the standard output of one command can be routed as the standard input of another command

```
$ command1 | command2
```

Pipelines (2)

- ▶ Example usage - We can use the less command to scroll through the output (page by page) of commands that send their results to standard output

```
$ ls -l /usr/bin | less
```

- ▶ What is the difference between a diversion and a pipeline?

```
command1 > file1
```

```
command1 | command2
```

- ▶ Don't use redirection with commands alone, as you can do a lot of damage.

Example

- ▶ `$ echo "cherry apple peach"`
`cherry apple peach`
- ▶ What if we want to sort these words in an alphabetical order?
- ▶ Step 1: The output from "echo" is the input in the translation command (tr) that will replace the " " line with a new line. ("`\n`"): `$ echo "cherry apple peach" | tr " " "\n"`
`cherry`
`apple`
`peach`
- ▶ Step 2: sorting `$ echo "cherry apple peach" | tr " " "\n" | sort`
`apple`
`cherry`
`peach`

Example

1. Let's try to reverse the order :

```
$ echo "cherry apple peach" | tr " " "\n" | sort -r
peach
cherry
apple
```

2. Normally, the output from the commands is printed on the screen. However, using the ">" symbol, we can redirect the output to a file:

```
$ date > RightNow.txt
$ cat RightNow.txt
Tue Dec 23 14:43:33 PST 2003
```

The above example used ">" to replace the content of any existing file having the name "RightNow.txt".

3. To append new data to an existing file, use ">>" instead:

```
$ date >> RightNow.txt
$ cat RightNow.txt
Tue Dec 23 14:43:33 PST 2003
Tue Dec 23 14:46:10 PST 2003
```

Remember: The ">" operator overwrites existing files, the ">>" operator appends existing files. In both cases, if the file does not exist, one is created.

Pipelines and Redirection (3)

1. Many commands have inputs as well as outputs. The input defaults to the keyboard, the output defaults to the screen.
2. To redirect the output to a file, use ">" or "»" as shown above.
3. To use the output of a command as an input to another command, use "|".
4. To use the contents of a file as an input to a command, use "<":

```
$ wc < RightNow.txt
```

```
2 12 58
```

As is so often the case in shell programming, there is at least one other way to produce the above result:

```
$ cat RightNow.txt | wc
```

```
2 12 58
```

Text filters

- ▶ Pipelines are often used for complex data processing. Commands used in this context are named as filters.
- ▶ Filters take the input, change it somehow, and then send it to the output.
- ▶ Example: Using the sort command, we can make a combined list of all programs in the /bin and /usr/bin directories and display it on the screen

```
$ ls /bin /usr/bin | sort | less
```

Command `uniq`

- ▶ Command *uniq* accepts a sorted list of data, either from standard input or from a single file, and removes all duplicates from the list.

```
$ ls /bin /usr/bin | sort | uniq | less
```

- ▶ Command *uniq* is usually used in conjunction with a command *sort*.
- ▶ To see which duplicates are in the list, use the option `-d`

```
$ ls /bin /usr/bin | sort | uniq -d | less
```

Command wc

- ▶ Command `wc` is used to display the number of lines, words, and bytes of files.
- ▶ Example:

```
$ wc ls-output.txt
```

```
7902 64566 503634 ls-output.txt
```

- ▶ If used without arguments, the `wc` command accepts standard input (keyboard) as input. (Press `ctrl+d` to quit)
- ▶ If we use the `-l` option, the command will only print the number of lines
- ▶ Example: Displays the number of units we have in the sorted list

```
$ ls /bin/usr/bin | sort | uniq | wc -l
```

Command grep

- ▶ grep is a powerful program used to find patterns in text files

```
grep pattern [datoteka...]
```

```
$grep 'text we are looking for' file-path
```

```
$grep -r 'text we are looking for' directory-path
```

```
$ grep 10001 zipcodes.txt
```

```
(print a line containing "10001")
```

```
$ man grep
```

- ▶ When the grep command encounters a pattern in a file, it prints the lines that contain it
- ▶ Example: We want to list all the files in our program list that contain the word *zip* in the program name

```
$ ls /bin /usr/bin | sort | uniq | grep zip
```

- ▶ option `-i`: greb ignores font size when searching
- ▶ option `-v`: prints lines that do not match the pattern

Commands head and tail

- ▶ The head command prints the first 10 lines of the text file

```
$ head ls-output.txt
```

- ▶ The tail command prints the last 10 lines of the text file

```
$ tail ls-output.txt
```

- ▶ The number of rows is adjusted using the `-n` number of rows option

```
$ head -n 5 ls-output.txt
```

```
$ tail -n 5 ls-output.txt
```

- ▶ Commands can be used in pipelines

```
$ ls /usr/bin | tail -n 5
```

- ▶ The tail command has a `-f` option to print files in real time. To end the printout, press CTRL `-c`

- ▶ Example: we can observe the process of writing operating system messages to log files (`/var/log/messages` or `/var/log/syslog`)

```
$ sudo tail -f /var/log/syslog
```

Commands find and tree

1. Command find searches a file with a specified name:

```
$ find . -name '*.jpg'
```

```
$ find . -iname 'fis*'
```

2. Command tree creates a text diagram of a file:

```
$ tree -d .
```

3. If command 'tree' does not exist, you may install it with the command:

```
$ sudo apt-get install tree
```

Command more

1. If the file is too long, we can use the more command to read the file:
`$ more zipcodes.txt`

Examining Files (2)

1. You can also use "grep" to print only those parts of a file you are interested in:

```
$ grep 10001 zipcodes.txt
```

(prints only those lines that have the character string "10001" in them)

The "grep" command is very useful, unfortunately it has a difficult-to-remember name. Be sure to:

```
$ man grep
```

There are a number of things you can do to find out more about the files in the list. Here are just a few:

2. The "file" command tries to identify files by examining their contents:

```
$ file tux_small.png tux_small.png: PNG image data, 128 x 151,  
8-bit/color RGB, non-interlaced
```

Examining Files (3)

1. Use the "cat" command to print the contents of a file:
`$ cat zipcodes.txt`
(prints the entire contents of a file named "zipcodes.txt")
2. If a file is too long to be viewed on one page, you can say:
`$ more zipcodes.txt`
(prints file one screenful at a time)
3. Use "grep" to print only those parts of a file you are interested in:
`$ grep 10001 zipcodes.txt`
(prints only those lines that have the character string "10001" in them)
4. The "grep" command is very useful, unfortunately it has a difficult-to-remember name. Try:
`$ man grep`

Exercise 1 (10 min.)

Enter the following commands in the Linux Terminal and try to interpret the output. Experiment a little, without fear :)

```
echo hello world
passwd
date
hostname
arch
uname -a
dmesg | more(you may need to press q to quit)
uptime
who am i
who
id
last
finger
w
top (you may need to press q to quit)
```

Exercise 2 (10 min.)

Enter the following commands in the Linux Terminal and try to interpret the output.

```
echo $SHELL
echo {con,pre}{sent,fer}{s,ed}
man "automatic door"
man ls (you may need to press q to quit)
man who (you may need to press q to quit)
who can tell me why i got divorced
lost
clear
cal 2000
cal 9 1752(do you notice anything unusual?)
bc -l(type quit or press Ctrl-d to quit)
echo 5+4 | bc -l
yes please(you may need to press Ctrl-c to quit)
time sleep 5
history
```

Using multiple commands at once

- ▶ So far we have seen how the command line interface is used to enter commands, we have looked at the results of command execution.
- ▶ You would like to be able to enter multiple commands at the same time
- ▶ You would also like to be able to process the results and the ability to pass the results of one command to the next
- ▶ Fulfillment of our wishes - shell script programming

What is a shell script?

- ▶ Shell script is a file that contains a set of commands.
- ▶ Shell reads the file and executes commands as if they were entered directly on the command line.
- ▶ strong command line interface to the system and scripting language interpreter at the same time
- ▶ automation of work in shell environment and simplification of tasks
- ▶ SH and BASH are the two most used shell environments

How do we write shell scripts?

1. Write a script

- ▶ shell scripts are plain text files
- ▶ are written using a text editor
- ▶ most editors use syntax highlighting
- ▶ file is saved with a .sh extension (example script.sh)

2. Make the script executable

- ▶ plain text files are not executable
- ▶ we need to change the file permissions using the chmod command

```
$ chmod + x script.sh
```

3. Saves the script to a location where the shell can be found

4. The following construction is used to execute the script

```
$ ./script.sh
```

Structure of the shell script

```
#!/bin/bash  
#First script  
echo 'Hello World!'
```

- ▶ The first command is always the specification of the environment in which the script will run

```
#!/bin/bash
```

- ▶ If this example does not work, you will need to find out where your Bash shell is located. Here's one on and how to find out:

```
$ whereis bash
```

- ▶ The second command is a comment. The `#` character is used
- ▶ Then the other commands follow.

Shell Script Basics

1. A shell script is a plain-text file that contains shell commands. It can be executed by typing its name into a shell, or by placing its name in another shell script.
2. To be executable, a shell script file must meet some conditions:

Structure of the shell script

1. A shell script file may optionally have an identifying suffix, like ".sh".
This only helps the user remember which files are which. The command processor responsible for executing the file uses the executable bit, plus the file's first line, to decide how to handle a shell script file.
2. Usually such a script is executed in this way:
\$./scriptname.sh
This special entry is a way to tell the command processor that the desired script is located in the current directory. Always remember: if you cannot get your shell script to run, remember this trick to provide its location as well as its name.

Task: Write your first Shell script

1. Select the text editor you want to use: gedit, nano, pizza, emacs, or you, or the X Windows editor, if you have this option.
2. Enter the following lines:

```
#!/bin/bash  
echo "Hello, world."
```
3. Save the file to the current working directory as "myscript.sh".
4. Move from the shell text editor.
5. At the command prompt, type:

```
$ chmod +x myscript.sh
```
6. To run the script, use the following command:

```
$ ./myscript.sh
```



```
Hello, world.
```

Message display – echo command

- ▶ In the previous example, we saw the echo command.

- ▶ The echo command is used to display a string

```
$ echo someString
```

- ▶ Example: use of special characters in the text

```
$ echo Let's see if this'll work
```

```
Lets see if thisll work
```

shell will ignore the ' character

- ▶ The use of double quotation marks is recommended

```
$ echo "Let's see if this'll work"
```

```
Let's see if this'll work
```

- ▶ However, if you need to use double quotes in the text itself, place a string of characters between single quotes

```
$ echo 'Rich says "scripting is easy"'
```

```
Rich says "scripting is easy"
```

Display metacharacters in the echo command

Sometimes we need to pass metacharacters to the command being run and do not want the shell to interpret them. There are three options to avoid shell interpretation of metacharacters.

- ▶ Most special metacharacters in shell scripts have a special meaning. Escape the metacharacter with a backslash.
- ▶ Use single quotes (' ') around a string. Single quotes protect all characters except the backslash.
- ▶ Use double quotes (" "). Double quotes protect all characters except the backslash, dollar sign (\$) and grave accent (').
- ▶ Metacharacters:

```
* ? [ ] ' "  
\ $ ; & (  
| ^ <>
```

- ▶ Proper use in shell scripts:

```
\* \? \[ \] \' \"  
\\ \$ \; \& \( \  
\\ \^ \< \>
```

TASK

Write a shell script to display the time and date and the users who are logged on to the system. Use the echo command.

```
#!/bin/bash
#time, date and users
echo Date and time is:
date
echo The following users are logged:
who
```

Variables

- ▶ As with all programming languages, the shell scripting language has variables.
- ▶ There is only one type of variable - a string
- ▶ Variable declaration does not exist.

User variables

- ▶ User variable names can be a string containing letters, numbers, and a `_` character.
- ▶ Names are case sensitive - `Var1` is not the same as `var1`
- ▶ You can assign a value to a variable using the `=` sign

`namevariable=quantityVariable`
- ▶ There must be no space between the variable name and the `=` sign, and between the `=` sign, and the value of the variable
- ▶ Examples:

`var1=10`
`var3=testing`
- ▶ The validity of user variables is as long as the script is executed.

Displays the value of a variable

- ▶ The value of the variable is obtained by using the \$ character before the variable name

- ▶ Example:

```
$var1
```

- ▶ We use large parentheses to display the value of a variable followed by a string

- ▶ Examples:

```
#!/bin/bash  
X=ABC  
echo "${X}abc"
```

```
#!/bin/bash  
num=2  
echo "This is ${num}nd track."
```

Redirecting the result of a command to a variable

- ▶ The output of the execution of a single command in a shell environment can be redirected to a variable
- ▶ TWO WAYS:

```
variable=$(command)  
variable='command'
```

- ▶ EXAMPLE:

```
#!/bin/bash  
myFile='ls /home/student'  
echo $myFile
```

Using structured commands

- ▶ So far we have written and used scripts in which commands were executed sequentially or in the order in which they are written
- ▶ Like other programming languages, shell has the option to change the order in which commands are executed
- ▶ Structures:
 - ▶ if-then
 - ▶ if-then-else
 - ▶ case
 - ▶ for-do
 - ▶ while-do
 - ▶ until-do

if-then-else

- ▶ The syntax of the if-then and if-then-else structures:
- ▶ Syntax

```
if command
then
    list of commands
fi
=====
if [ condition ]
then
    block_of_statements
else
    block_of_statements
fi
```

- ▶ The meaning of the if-then-else structure in the bash shell is following:
 - ▶ Bash shell executes a command defined by the word *if*
 - ▶ If the output state of the command is zero, or if the command is successfully executed, the list of commands following the word then is executed
 - ▶ If the output state of the command is non-zero, or if the command is not executed successfully, the list of commands following the word else are executed

Tests and Branching

1. Bash shell scripts can perform, and act on, various kinds of tests, and here is the most basic introduction.
2. To follow these and other examples that involve multiline shell scripts, please set up to edit and run a test script file (let's call it "myscript.sh").
3. Here is an example of a test and branch:

```
if [ -e . ]  
then  
    echo "Yes."  
else  
    echo "No."  
fi
```

Run the test script:

```
$ ./myscript.sh  
Yes.
```

NOTE. We created a test (the part of the script between "[" and "]") which tested whether a particular element existed ("-e"). Because the symbol "." in this context means the current directory, the test succeeded. Try replacing the "." with something that is not present in the current directory, example "xyz". See how the outcome changes. It is important to realize that "[" is an alias for the command "test".

- └ Using structured commands
 - └ The structure if-then-else

Example

```
if test -e .  
then  
    echo "Yes."  
else  
    echo "No."  
fi
```

Do not forget to read "test" man page to find out all the different tests that are available: `$ man test`

```
file="/etc/passwd"  
if [ -e $file ]  
then  
    echo "whew"  
else  
    echo "uh-oh"  
fi
```

Loops and repetitions

Here are some examples of loop operators:

```
for fn in *; do
    echo "$fn"
done
```

In this example, the "*" is expanded by the shell to a list of all the files in the current directory, then each filename is applied to the loop control area. In such a construct, any whitespace-delimited list will do:

```
for fn in tom jack harry; do
    echo "$fn"
done
```

```
$ ./myscript.sh
tom
james
harry
```

This method will work for any list that uses spaces as delimiters. But what happens if you must parse a list that must be delimited by linefeeds instead of spaces, such as the case of a list of filenames or paths that contain spaces as part of their names? You can solve such a problem this way (there are other solutions):

```
ls -1 | while read fn; do
    echo "$fn"
done
```

This example uses an option to "ls" (note: the option is "-" followed by the numerical digit "1", not a lowercase "l") that formats file listings with one name per line, then this list is pipelined to a routine that reads lines until there are no more to read. This meets the requirement that linefeeds become the delimiters between list elements, not spaces. There is plenty more to this topic. Please refer to the references of for more.

- └ Using structured commands
- └ The structure if-then-else

Using Numbers in Scripts

Numbers can easily be accommodated in scripts. Example:

```
n=1
while [ $n -le 6 ]; do
    echo $n
    let n++
done
$ ./myscript.sh
1
2
3
4
5
6
```

Notice the "let" command, which treats its argument in a way meant to accommodate numbers.

Secure Shell SSH

1. SSH solves two basic problems of secure communication with a remote host. Firstly, it authenticates that the remote host is the one that says it is (and hence prevents the so-called "man in the middle" attacks), and secondly it encrypts all communications between local and remote hosts.
2. SSH consists of two parts. The SSH server runs on the remote host, and listens for incoming connections on port 22, while the SSH client is used on the local system to communicate with the remote server.

Simple password-based login and SSH hopping

1. Simple password-based login: `ssh user@server1.domain.si`
2. SSH hopping
 - ▶ `$ ssh <userID>@server1.domain.si`
 - ▶ `user_server1$ ssh <userID>@server2.domain.si`
 - ▶ `user_server2$`
3. Newer versions of OpenSSH have the so-called 'ProxyJump' option `-J` (the same example above would be)
 - ▶ `$ ssh -J <userID>@server1.domain.si, user_server2`

Copying data

Scenario: A user uses two Linux machines called A and B. She is logged on the machine A and through the SSH she connects to the machine B.

1. How can she copy a file from System B to System A?
2. How can she copy a file from System A to System B?

Syntax:

`scp <source> <destination>`

1. copy a file from B to A while logged into B:
`scp /path/to/file username@a:/path/to/destination`
2. To copy a file from B to A while logged into A:
`scp username@b:/path/to/file /path/to/destination`

Public key authentication

1. Creation of keys: `ssh-keygen`
2. Distribution of Keys: `ssh-copy-id -i ~/.ssh/mykey user@host`
(`cd ~/.ssh`)
3. Using the keys: `ssh -i ~/.ssh/mykey user@host`

1. [http://en.wikipedia.org/wiki/Bash_\(Unix_shell\)](http://en.wikipedia.org/wiki/Bash_(Unix_shell))
2. http://www.arachnoid.com/linux/shell_programming.html
3. Predavanja in vaj po operacijskih sistemov, FIŠ.
4. <https://waterprogramming.wordpress.com/category/tutorials/tips-and-tricks/>
5. <https://waterprogramming.wordpress.com/category/programming/bash/>
6. <https://waterprogramming.wordpress.com/2012/01/12/the-cluster-and-basic-unix-commands/>
7. <https://doku.lrz.de/display/PUBLIC/SSH+Tutorial>
8. <https://doku.lrz.de/display/PUBLIC/Linux+and+Shell+Tutorial>